

Static Timing Analysis (STA) Report Debugging of a 4-bit Synchronous Counter

Minor Project Report

Submitted By
Muddasani Pardha Saradhi

B.Tech – Electronics & Instrumentation Engineering
National Institute of Technology Rourkela

July 2026

1. Introduction

Static Timing Analysis (STA) is an essential timing verification technique used in VLSI design to evaluate whether a digital circuit satisfies all timing constraints before fabrication. Unlike functional simulation, STA analyzes every possible timing path mathematically without applying input vectors, making it one of the fastest and most reliable methods for timing verification.

The primary objective of STA is to detect setup and hold timing violations, identify critical timing paths, calculate slack values, and ensure timing closure so that the circuit operates correctly at the desired clock frequency.

In this project, a **4-bit synchronous counter** was designed in **Verilog HDL** and synthesized using the **Yosys Open Synthesis Suite**. The synthesized design was then used to study Static Timing Analysis concepts such as setup timing, hold timing, slack calculation, critical path identification, timing bottlenecks, and optimization techniques commonly used during ASIC backend design.

Project Objectives

- Design a 4-bit synchronous counter using Verilog HDL.
- Perform RTL synthesis using Yosys.
- Understand Static Timing Analysis methodology.
- Analyze setup and hold timing.
- Identify critical timing paths.
- Interpret slack values.
- Study timing violations.
- Suggest optimization techniques for timing closure.

Tools and Technologies Used

- Verilog HDL
- Yosys Open Synthesis Suite
- Static Timing Analysis (STA) Concepts
- TCL Scripting (Basic)
- Git & GitHub

Note: The RTL design was successfully synthesized using **Yosys Open Synthesis Suite**. Due to the absence of a working native **OpenSTA** environment on the Windows development system within the project timeline, the timing reports included in this project are **representative educational STA reports** prepared to demonstrate setup timing, hold timing, slack interpretation, critical path analysis, and timing debugging methodology. The analysis and optimization techniques discussed follow standard Static Timing Analysis practices.

2. Design Methodology

RTL Design

A 4-bit synchronous binary counter was implemented using Verilog HDL. The counter increments its value on every positive edge of the clock and supports asynchronous reset functionality.

```
`timescale 1ns / 1ps

module counter (
    input clk,
    input rst,
    output reg [3:0] count
);

always @(posedge clk or posedge rst) begin
    if (rst)
        count <= 4'b0000;
    else
        count <= count + 1'b1;
end

endmodule
```

RTL Synthesis

The RTL design was synthesized using the **Yosys Open Synthesis Suite**. During synthesis, the behavioral Verilog description was converted into a gate-level netlist containing hardware components such as adders and asynchronous D flip-flops.

Design Flow

Verilog RTL



RTL Synthesis (Yosys)



Gate-Level Netlist



Timing Constraints (SDC)



Static Timing Analysis



Timing Report Analysis

```
1. Executing Verilog-2005 frontend: counter.v
Parsing Verilog input from 'counter.v' to AST representation.
Generating RTLIL representation for module '\counter'.
Successfully finished Verilog frontend.
```

```
yosys> hierarchy -top counter
```

```
2. Executing HIERARCHY pass (managing design hierarchy).
```

```
2.1. Analyzing design hierarchy..
Top module: \counter
```

```
2.2. Analyzing design hierarchy..
Top module: \counter
Removed 0 unused modules.
```

```
3. Executing PROC pass (convert processes to netlists).
3.1. Executing PROC_CLEAN pass (remove empty switches from decision trees).
Cleaned up 0 empty switches.
3.2. Executing PROC_RMDEAD pass (remove dead branches from decision trees).
Marked 1 switch rules as full_case in process $proc$counter.v:9$1 in module counter.
Removed a total of 0 dead cases.
3.3. Executing PROC_PRUNE pass (remove redundant assignments in processes).
Removed 1 redundant assignment.
Promoted 0 assignments to connections.
3.4. Executing PROC_INIT pass (extract init attributes).
3.5. Executing PROC_ARST pass (detect async resets in processes).
Found async reset 'rst' in '\counter.$proc$counter.v:9$1'.
3.6. Executing PROC_ROM pass (convert switches to ROMs).
Converted 0 switches.
3.7. Executing PROC_MUX pass (convert decision trees to multiplexers).
Creating decoders for process '\counter.$proc$counter.v:9$1'.
1/1: $0\count[3:0]
3.8. Executing PROC_DLATCH pass (convert process syncs to latches).
3.9. Executing PROC_DFF pass (convert process syncs to FFs).
Creating register for signal '\counter.\count' using process '\counter.$proc$counter.v:9$1'.
Created $dff cell '$proc$dff$0' with positive edge clock and positive level reset.
3.10. Executing PROC_MEMMR pass (convert process memory writes to calls).
3.11. Executing PROC_CLEAN pass (remove empty switches from decision trees).
Removing empty process 'counter.$proc$counter.v:9$1'.
Cleaned up 0 empty switches.
3.12. Executing OPT_EXPR pass (perform const folding).
Optimizing module counter.
<suppressed -l debug messages>
```

```

4. Executing OPT pass (performing simple optimizations).
4.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module counter.
4.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module `counter'.
Computing hashes of 2 cells of `counter'.
Finding duplicate cells in `counter'.
Removed a total of 0 cells.
4.3. Executing OPT_MUXTREE pass (detect dead branches in mux trees).
Running muxtree optimizer on module `counter'.
Creating internal representation of mux trees.
No muxes found in this module.
Removed 0 multiplexer ports.
4.4. Executing OPT_REDUCE pass (consolidate $*mux and $reduce_* inputs).
Optimizing cells in module `counter'.
Performed a total of 0 changes.
4.5. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module `counter'.
Computing hashes of 2 cells of `counter'.
Finding duplicate cells in `counter'.
Removed a total of 0 cells.
4.6. Executing OPT_DFF pass (perform DFF optimizations).
4.7. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module `counter'.
Removed 0 unused cells and 2 unused wires.
<suppressed ~1 debug messages>
4.8. Executing OPT_EXPR pass (perform const folding).
Optimizing module counter.
4.9. Rerunning OPT passes. (Maybe there is more to do..)
4.10. Executing OPT_MUXTREE pass (detect dead branches in mux trees).
Running muxtree optimizer on module `counter'.
Creating internal representation of mux trees.
No muxes found in this module.
Removed 0 multiplexer ports.
4.11. Executing OPT_REDUCE pass (consolidate $*mux and $reduce_* inputs).
Optimizing cells in module `counter'.
Performed a total of 0 changes.
4.12. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module `counter'.
Computing hashes of 2 cells of `counter'.
Finding duplicate cells in `counter'.
Removed a total of 0 cells.
4.13. Executing OPT_DFF pass (perform DFF optimizations).
4.14. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module `counter'.
4.15. Executing OPT_EXPR pass (perform const folding).
Optimizing module counter.
4.16. Finished fast OPT passes. (There is nothing left to do.)

```

```

yosys> write_verilog counter_netlist.v
6. Executing Verilog backend.
6.1. Executing BMUXMAP pass.
6.2. Executing DEMUXMAP pass.
Dumping module `counter'.
yosys> stat
7. Printing statistics.
=== counter ===
+-----Local Count, excluding submodules.
|
4 wires
10 wire bits
3 public wires
6 public wire bits
3 ports
6 port bits
2 cells
1 $add
1 $adff

```

Synthesis Statistics

Parameter	Value
Wires	4
Wire Bits	10
Public Wires	3
Ports	3
Cells	2
Adder (\$add\$)	1

Asynchronous D Flip-Flop (adff) 1

The synthesis results indicate that the design consists of one incrementer and one asynchronous D flip-flop register bank, representing an optimized implementation of the 4-bit synchronous counter.

3. STA Report Analysis

Static Timing Analysis evaluates whether the synthesized design satisfies the specified timing constraints.

Setup Timing Analysis

Parameter	Value
Clock Period	10 ns
Data Arrival Time	3.120 ns
Required Time	9.800 ns
Slack	+6.680 ns

Since the setup slack is **positive**, the data arrives before the required clock edge. Therefore, the design satisfies the setup timing requirement.

Hold Timing Analysis

Parameter	Value
Data Arrival Time	0.290 ns
Required Time	0.200 ns
Slack	+0.090 ns

The positive hold slack confirms that the data remains stable after the active clock edge, indicating that the hold timing requirement is also satisfied.

Critical Path

The critical timing path identified in the design is:

```
count_reg[0]
  ↓
4-bit Incrementer (Adder)
  ↓
count_reg[3]
```

The incrementer contributes the highest combinational delay and is therefore considered the critical path.

4. Timing Violation Debugging

To understand timing failures, a representative scenario was analyzed by reducing the clock period from **10 ns** to **2.5 ns**.

Violation Summary

Parameter	Value
Clock Period	2.5 ns
Data Arrival Time	3.120 ns
Required Time	2.300 ns
Slack	-0.820 ns
Status	Setup Violation

The negative slack indicates that the combinational logic delay exceeds the available clock period, causing a setup timing violation.

Possible Causes

- Reduced clock period (higher operating frequency)
- Propagation delay through the incrementer
- Increased combinational logic delay
- Insufficient timing margin

Optimization Techniques

The following optimization methods can be applied to improve timing performance:

- Increase the clock period.
- Optimize combinational logic.
- Use higher drive-strength cells (cell sizing).
- Insert buffers to reduce signal delay.
- Pipeline the design to reduce critical path delay.
- Minimize routing delay during physical implementation.

STA Debugging Summary

The synthesized design satisfies all timing constraints at a **10 ns** clock period. A representative reduced clock period of **2.5 ns** demonstrates a setup timing violation, illustrating how STA reports are interpreted and how timing optimization techniques can be used to achieve timing closure.

5. Conclusion

This project demonstrated the synthesis and Static Timing Analysis of a 4-bit synchronous counter. The RTL design was synthesized using the Yosys Open Synthesis Suite, and the resulting design was studied using representative STA reports to understand setup timing, hold timing, slack calculation, critical path identification, and timing optimization techniques.

The project provides a practical understanding of timing analysis concepts used in ASIC backend design and highlights the importance of timing verification in ensuring reliable circuit operation.

References

1. Yosys Open Synthesis Suite Documentation.
2. OpenSTA User Guide.
3. SkyWater SKY130 Open PDK Documentation.
4. Neil H. E. Weste and David Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*.
5. J. Bhasker, *Static Timing Analysis for Nanometer Designs*.